**Title:**
Global Video Game Sales & Market Trends Explorer

**Team Members:**
Josh Stewart and Lucci Montes

**Data Set URL and Attributes:**
https://www.kaggle.com/datasets/kendallgillies/video-game-sales-and-ratings

The fields include:

- Name - The game's name
- Platform - Platform of the games release
- Year_of_Release - Year of the game's release
- Genre - Genre of the game
- Publisher - Publisher of the game
- NA_Sales - Sales in North America (in millions)
- EU_Sales - Sales in Europe (in millions)
- JP_Sales - Sales in Japan (in millions)
- Other_Sales - Sales in the rest of the world (in millions)
- Global_Sales - Total worldwide sales (in millions)
- Critic_score - Aggregate score compiled by Metacritic staff
- Critic_count - The number of critics used in coming up with the critic score
- User_score - Score by Metacritic's subscribers
- User_count - Number of users who gave the user score
- Rating - The ESRB ratings

**Problem/Motivation:**

The modern video game industry generates vast amounts of historical sales data spanning multiple geographic regions, platforms, and genres. However, raw datasets often exist as large, unorganized CSV files that are difficult for analysts, consumers, or enthusiasts to interpret without programming skills or dedicated business intelligence tools.

**Motivation:**

Understanding market trends, such as regional preference variations (e.g., North America vs. Japan), platform longevity, and publisher performance over time, is critical for decision-making in interactive media. By building our project, we bridge this gap with a desktop application that transforms raw tabular data into accessible visual charts and parameterized analytical queries. This project directly addresses key computational challenges: efficiently processing thousands

of records, separating data handling from analytical logic, validating user parameters, and delivering interactive, exportable summaries.

**Research Questions:**

Top-10 Ranking: What are the top *10* highest-selling games or bottom 10 in a user-selected Genre and Region? *(Approach: Top-k / Bottom-k ranking with user parameter $k$)*

Comparison Across Categories: How do total global sales compare across platforms for a user-specified Release Year range? Or how do these sales differ by region? *(Approach: Categorical comparison)*

Growth / Change Over Time: How have sales for a user-selected Genre trended over a specified decade? How have the sales changed as platforms have updated? *(Approach: Time-series / trend analysis)*

Distribution & Summary: What are the mean, median, and interquartile range of sales across all regions for a selected Publisher? *(Approach: Statistical summary)*

## Anticipated Analyses and Charts

### 1. Top-$k$ Ranking Analysis

- **Analytical Approach:** Ranking / Sorting with Parameterized Limit ($k$) and Filtering.
- **Algorithm & Method:** Filter records by the user-selected Genre and Region. Compute sorting indices on the corresponding sales column (e.g., naSales, euSales, jpSales, globalSales) using a quicksort or custom Java Comparator. Extract the top $k$ records while handling tie conditions gracefully.
- **Anticipated Visual Chart: Horizontal Bar Chart** (via JFreeChart / XChart).
    - *X-Axis:* Sales Revenue (in millions of USD).
    - *Y-Axis:* Game Title.
    - *Purpose:* Displays comparative revenue of top-performing titles side-by-side.

### 2. Cross-Category Platform Comparison

- **Analytical Approach:** Category Comparison & Aggregation across Numeric Range.
- **Algorithm & Method:** Filter dataset by Release Year within $[StartYear, EndYear]$. Aggregate globalSales grouped by Platform using a Map<String, Double>. Sort platforms by total revenue to identify dominant hardware platforms during that period.
- **Anticipated Visual Chart: Grouped / Stacked Bar Chart**.
    - *X-Axis:* Platform (e.g., PS4, XOne, Switch, PC).
    - *Y-Axis:* Total Global Revenue (Millions USD).
    - *Purpose:* Compares market share and platform performance within the chosen timeframe.

### 3. Longitudinal Time-Series Trend

- **Analytical Approach:** Growth and Change Over Time (Time-Series Analysis).
- **Algorithm & Method:** Filter records matching the user-selected Genre. Group and sum globalSales by Release Year across the requested year span into a sorted TreeMap<Integer, Double>. Calculate year-over-year percentage change.
- **Anticipated Visual Chart: Line Chart / Trend Line**.
    - *X-Axis:* Release Year (Timeline).
    - *Y-Axis:* Total Annual Sales Revenue (Millions USD).
    - *Purpose:* Illustrates the historical rise, peak, and decline of specific gaming genres over time.

### 4. Statistical Distribution Summary

- **Analytical Approach:** Statistical Measures (Mean, Median, Interquartile Range).
- **Algorithm & Method:** Extract global sales values for all games published by the user-selected Publisher. Sort the values to calculate statistical metrics:

$$Mean = \frac{\sum Sales}{N}, Median = Q_2, IQR = Q_3 - Q_1$$

- **Anticipated Visual Chart: Box Plot** (or Summary Metric Cards alongside a Distribution Bar Chart).

    - *X-Axis:* Selected Publisher.
    - *Y-Axis:* Sales Revenue per Game.
    - *Purpose:* Visualizes average performance, variance, and commercial outliers (hit titles vs. average releases) for a publisher.

**5. Multi-Criteria Filtered Aggregation**

- **Analytical Approach:** Parameterized Multi-Criteria Threshold Filtering & Grouping.
- **Algorithm & Method:** Filter dataset by Genre. Group records by Publisher and count total releases. Filter out publishers where $GameCount < N$. Compute average global revenue per game for the qualifying publishers ($\frac{TotalSales}{GameCount}$).
- **Anticipated Visual Chart: Clustered Bar Chart**.
    - *X-Axis:* Publisher Name.
    - *Y-Axis:* Average Revenue per Released Game (Millions USD).
    - *Purpose:* Highlights high-efficiency publishers who achieve high average sales despite specified output thresholds.

https://github.com/joshstewart805/CSC-220-Final-Project

**Description:**

`Game` **(Data Model):** Represents a single video game record, holding attributes like title, genre, release year, platform, publisher, and regional sales figures.

**Code Framework:**

```
class Game {
      - int rank
      - String name
      - String platform
      - int year
      - String genre
      - String publisher
      - double naSales
      - double euSales
      - double jpSales
      - double otherSales
      - double globalSales
      + Game(int, String, String, int, String, String, double, double, double, double, double)
      + getRank() int
      + getName() String
      + getPlatform() String
      + getYear() int
      + getGenre() String
      + getPublisher() String
      + getNaSales() double
      + getEuSales() double
      + getJpSales() double
      + getOtherSales() double
      + getGlobalSales() double
      + toString() String
  }
```

**Description:**

`PublisherStats` **(Data Transfer Object):** Stores calculated statistical summary values—such as mean, median, interquartile range, and total game count—specifically for publisher performance analysis.

**Code Framework:**

```
class PublisherStats {
    - String publisher
    - double meanSales
    - double medianSales
    - double iqrSales
    - int totalGames
    + PublisherStats(String, double, double, double, int)
    + getPublisher() String
    + getMeanSales() double
    + getMedianSales() double
    + getIqrSales() double
    + getTotalGames() int
}
```

**Description:**

`DataLoader` **(Data Access Layer):** Reads the raw CSV dataset, validates and cleans incomplete records, and parses the rows into Java objects.

**Code Framework:**

```
class DataLoader {
    - String filePath
    + DataLoader(String)
    + loadCSV(String) List~Game~
    - parseRow(String) Game
    - isValidRow(String[]) boolean
}
```

**Description:**

`AnalysisService` **(Business & Analytics Engine):** Executes all calculation algorithms, filtering logic, and parameterized analytical queries independently of the user interface.

**Code Framework:**

```
class AnalysisService {
    - List~Game~ dataset
    + AnalysisService(List~Game~)
    + getTopKGames(String region, String genre, int k) List~Game~
    + getSalesByPlatform(int startYear, int endYear) Map~String, Double~
    + getGenreTrend(String genre, int startYear, int endYear) Map~Integer, Double~
    + getPublisherStats(String publisher) PublisherStats
    + getTopPublishersByGenre(String genre, int minGames) Map~String, Double~
}
```

**Description:**

`ChartFactoryManager` **(Visualization Layer):** Converts computed analytical results into interactive visual charts (such as bar charts, line graphs, and box plots) using a charting library.

**Code Framework:**

```
class ChartFactoryManager {
    + createTopKChart(List~Game~ games, String title) ChartPanel
    + createPlatformComparisonChart(Map~String, Double~ data) ChartPanel
    + createGenreTrendChart(Map~Integer, Double~ data, String genre) ChartPanel
    + createPublisherStatsChart(PublisherStats stats) ChartPanel
    + createPublisherAvgRevenueChart(Map~String, Double~ data) ChartPanel
}
```

**Description:**

`CSVExporter` **(File I/O Component):** Takes the currently computed analysis results from the table and writes them to a user-selected CSV file.

**Code Framework:**

```
class CSVExporter {
    + exportAnalysisToCSV(File targetFile, String[] headers, List~String[]~ dataRows) void
}
```

**Description:**

`MainFrame` **(User Interface Presentation):** Renders the graphical user interface, manages input controls (spinners, dropdowns, buttons), handles user actions, and displays tabular results alongside generated charts.

**Code Framework:**

```
class MainFrame {
    - AnalysisService analysisService
    - JComboBox~String~ questionSelector
    - JPanel controlPanel
    - JPanel chartContainerPanel
    - JTable resultTable
    - JButton btnRunAnalysis
    - JButton btnReset
    - JButton btnExportCSV
    + MainFrame(AnalysisService)
    - initGUI() void
    - updateParameterControls(int questionIndex) void
    - executeAnalysis() void
    - exportResults() void
}
```